SIMATS ENGINEERING

Assessment Tool 2 - Data Interpretation (CO-1)

Course: Internet Programming | Course Code: CSA43

Question 1: HTTP Response Analysis

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

1. Incorrect or invalid fields

- **Content-Length: -150** is invalid. The value must be a non-negative integer giving the body size in bytes.
- **Date** has a wrong day name. 01 April 2026 falls on a Wednesday, not Monday, so the header is inconsistent.
- The status line, Content-Type and Connection fields are syntactically valid.

2. Issue with Content-Length and its impact

Content-Length tells the client exactly how many bytes of body to read. A negative number is meaningless, so a standards-compliant client treats the message as malformed. The browser may reject the response, show a blank or partial page, or wait until the connection times out. Because the connection is *keep-alive,* the client cannot tell where this response ends and the next begins, so the connection cannot be reused safely. Wrong length values can also lead to request/response smuggling attacks.

3. Does the status code match the response?

Not really. **200 OK** claims the request succeeded and the message is well-formed, but the header is corrupted. Either the server generated a faulty length (a server-side bug that is better reported as **500 Internal Server Error**) or the response must be fixed so that 200 is truthful. The status code is acceptable only after the headers are corrected.

4. Suggested corrections

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 150
Connection: keep-alive
```

Here 150 stands for the real byte size of the body. If the size is not known beforehand, the server should drop Content-Length and use **Transfer-Encoding: chunked** instead. The server should also compute the length automatically rather than hard-coding it.

Question 2: DOM Structure Analysis

```
<html>
 <head>
  <title>Test Page</title>
 </head>
 <body>
  <div> <p>Hello World
  </div>
 </body>
</html>
```

1. Missing or improperly closed elements

- The ** ** element has no closing tag .
- The **<!DOCTYPE html>** declaration is missing at the top of the document.
- The <html>, <head>, <title>, <body> and tags are properly opened and closed.

2. How the browser interprets the structure

HTML is parsed leniently. The end tag of is optional, so when the parser meets it automatically closes the open paragraph first and then the div. The resulting DOM tree is: **html → (head → title) and (body → div → p → text "Hello World")**. The browser does not stop or show an error. Without a DOCTYPE, however, it switches to *quirks mode*. If the same text were parsed as XHTML/XML, it would be rejected as not well-formed because the p element is never closed.

3. Impact on rendering and layout

In this small page the visible output is the same: the text "Hello World" in a paragraph inside a div. The risk is in larger pages. An unclosed element can make later content nest inside it unexpectedly, which changes margins, box sizes and CSS selector matching. Quirks mode also alters the box model and default font sizes, so the layout may differ between browsers. Scripts that walk the DOM can also get a different tree than the author intended.

4. Corrections to make the DOM valid

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8">
  <title>Test Page</title>
 </head>
 <body>
  <div>
   <p>Hello World</p>
  </div>
 </body>
</html>
```

Close every element explicitly and add the DOCTYPE and charset.

Question 3: GET vs POST in Login System

1. Differences in data transmission

Aspect	GET	POST
Where data goes	Appended to the URL as a query string	Sent inside the request body
Visibility	Visible in address bar, history, logs	Not shown in the URL
Length limit	Limited by URL length	Practically no limit
Caching / bookmarks	Can be cached and bookmarked	Not cached or bookmarked by default
Intended use	Safe retrieval of data	Submitting data / changing state
Security level	Low	Moderate (needs HTTPS)

2. Which method is more suitable for login

POST is the suitable method. Credentials travel in the body, so they do not appear in the address bar, browser history or referrer header, and the request is not cached or bookmarked. GET is meant for harmless, repeatable reads and must never carry secrets.

3. Security risks of using GET for authentication

- The password (*/login?user=admin&pass=1234*) is stored in browser history and bookmarks.
- Web server, proxy and firewall logs record the full URL with the password.
- The URL can leak to other sites through the Referer header.
- Shoulder surfing and shared links expose the credentials; cached copies may keep them.
- It encourages replay and cross-site request forgery because the login can be triggered by a plain link.

4. Best practices for secure login

- Use **POST over HTTPS/TLS** only; redirect HTTP to HTTPS and enable HSTS.
- Never store plain passwords; hash with bcrypt, scrypt or Argon2 and a unique salt.
- Add CSRF tokens, rate limiting, account lockout and CAPTCHA against brute force.
- Set session cookies with *HttpOnly, Secure, SameSite* and regenerate the session ID after login.
- Show generic error messages ("invalid credentials"), validate input, and offer multi-factor authentication.

Note: POST by itself does not encrypt data; it only hides it from the URL. Encryption comes from HTTPS.

Question 4: HTML Code Inspection

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Missing closing tags

- **</h1>** is missing after "Welcome".
- ** ** is missing after "This is a paragraph".
- <img> is a void element, so it needs no closing tag in HTML5 (in XHTML it must be written).

2. Issues with the <img> tag

- The required **alt** attribute is missing. It matters for accessibility, SEO and for text shown when the image fails to load.
- No width and height are given, which can cause layout shifts while the image loads.
- Because it sits inside the unclosed heading and paragraph, it is nested in the wrong place.

3. How browsers handle these errors

The HTML5 parser recovers silently. A start tag does not close an open <h1>, so the paragraph and the image become *children of the h1*. The whole paragraph is therefore shown in large bold heading style. Everything stays open until </body> arrives, where the parser closes all pending elements automatically. No error appears, but the structure differs from what the author meant, and an XHTML parser would refuse the page completely.

4. Corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8">
 <title>Sample</title>
</head>
<body>
 <h1>Welcome</h1>
 <p>This is a paragraph</p>
 <img src="image.jpg" alt="Sample image" width="300" height="200">
</body>
</html>
```

Question 5: Browser-Server Communication Flow

1. Interpretation of each step, and 2. where delays can occur

Step	What happens	Possible delay
1. URL entered	Browser parses scheme, host, path; checks cache and HSTS list.	Minimal; slow only with redirects.
2. DNS lookup	Domain name is resolved to an IP address.	Slow or distant DNS server, uncached lookup.
3. HTTP request	TCP (and TLS) connection is made, then GET request is sent.	Network latency, handshakes, packet loss.
4. Server processes	Server runs code, queries the database, builds the page.	Heavy scripts, slow queries, overloaded server.
5. Response returned	Status line, headers and body travel back.	Large files, low bandwidth, no compression.
6. Browser renders	HTML parsed to DOM, CSS and JS loaded, page painted.	Render-blocking CSS/JS, many images.

3. Role of DNS

DNS works like the internet's phone book. Browsers need an IP address to contact a server, so DNS translates the human-readable domain name (e.g. www.example.com) into that address. Lookups go through the browser and OS caches, then the resolver, root, TLD and authoritative servers. Without DNS no connection can start, and a slow lookup delays every first visit to a site.

4. Optimizations to improve performance

- Use DNS caching, a fast DNS provider and *dns-prefetch* hints; keep TTL values sensible.
- Reuse connections with keep-alive, HTTP/2 or HTTP/3, and enable TLS session resumption.
- Use a CDN and browser caching (Cache-Control, ETag) to serve content closer to the user.
- Enable Gzip/Brotli compression; minify HTML, CSS and JS; optimize and lazy-load images.
- Optimize server code and database queries, and add server-side caching.
- Reduce redirects and the number of requests; load scripts with async/defer.

Conclusion: total page-load time is the sum of all six steps, so tuning the slowest step gives the biggest gain.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____